



Desenvolvimento de Serviços com Spring Boot

Etapa/Aula 06.1

Professor(a): Flávio Neves

E-mail: flavio.neves@prof.infnet.edu.br

Configurações de Serviços com Redis, JPA e MongoDB

Roteiro da Aula

- **SpringBoot Actuator**
- **Configurações de Serviços com Redis**

SpringBoot Actuator

- Além das funcionalidades principais para desenvolver aplicações, o Spring Boot também fornece um conjunto de funcionalidades adicionais para o suporte operacional da sua aplicação.
- Uma aplicação é considerada operacional quando está em produção e a servir os seus clientes ou utilizadores.
- Para gerir um serviço sem falhas para os seus clientes, é necessário monitorizar e gerir a sua aplicação.

SpringBoot Actuator

- Esta monitorização e gestão inclui a compreensão do estado da aplicação, do desempenho, do tráfego de entrada e de saída, da auditoria, de várias métricas da aplicação (mais sobre isto adiante), do reinício da aplicação, da alteração do nível de registo da aplicação e muito mais.
- O monitoramento dos detalhes das métricas permitem analisar o comportamento da aplicação e agir com base nas necessidades.

SpringBoot Actuator

- O Spring Boot actuator traz esses recursos de monitoramento e gerenciamento para seu aplicativo Spring Boot.
- O principal benefício do Spring Boot Actuator é que você obtém muitos recursos prontos para produção em seu aplicativo sem implementá-los explicitamente em seu aplicativo.

SpringBoot Actuator

- O actuator endpoint permite monitorar e gerir a sua aplicação, e permite monitorar o status de saúde da aplicação.
- O Spring Boot fornece vários endpoints internos que podem ser usados imediatamente.
- Também pode adicionar o seu endpoint personalizado específico para a sua aplicação.

SpringBoot Actuator

- Os endpoints do actuator podem ser acessados através da Web (HTTP) ou JMX (Java Management Extensions), e pode ativar, desativar ou expor os endpoints.
- As opções ativadas ou desativadas indicam que pode controlar a permissão de um endpoint do actuator específico na aplicação.

SpringBoot Actuator

- Por exemplo, por predefinição, o endpoint de encerramento que permite encerrar uma aplicação em execução está desativado por motivos de segurança.
- Você pode substituir este comportamento predefinido e ativá-lo na sua aplicação.
- A opção expor indica se um endpoint específico está exposto para ser acessado através de um modo de acesso (por exemplo, através de HTTP ou JMX).



Project

☐ Gradle - Groovy

☐ Gradle - Kotlin ☒ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 3.3.0 (SNAPSHOT) ☐ 3.3.0 (RC1) ☐ 3.2.6 (SNAPSHOT) ☒ 3.2.5

☐ 3.1.12 (SNAPSHOT) ☐ 3.1.11

Project Metadata

Group

Artifact

Name

Description

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot Actuator OPS

Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.

Arquivo Editar Exibir Histórico Favoritos Ferramentas Ajuda

localhost:8080/actuator

← → ↻ localhost:8080/actuator

Introdução :: Pessoal ADM Online ...

JSON Dados brutos Cabeçalhos

Salvar Copiar Recolher tudo Expandir tudo Filtrar JSON

```
▼ _links:
  ▼ self:
    href:      "http://Localhost:8080/actuator"
    templated: false
  ▼ health:
    href:      "http://Localhost:8080/actuator/health"
    templated: false
  ▼ health-path:
    href:      "http://Localhost:8080/actuator/health/{*path}"
    templated: true
```

SpringBoot Actuator

- <https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html>

SpringBoot Actuator

- Pode ser utilizado para selecionar todos os endpoints.
- Por exemplo, para expor tudo através de HTTP, exceto os endpoints env e beans, utilize as seguintes propriedades:

Properties

Yaml

```
management.endpoints.web.exposure.include=*  
management.endpoints.web.exposure.exclude=env,beans
```

localhost:8080/actuator

localhost:8080/actuator

Introdução :: Pessoal ADM Online ...

JSON Dados brutos Cabeçalhos

Salvar Copiar Recolher tudo Expandir tudo Filtrar JSON

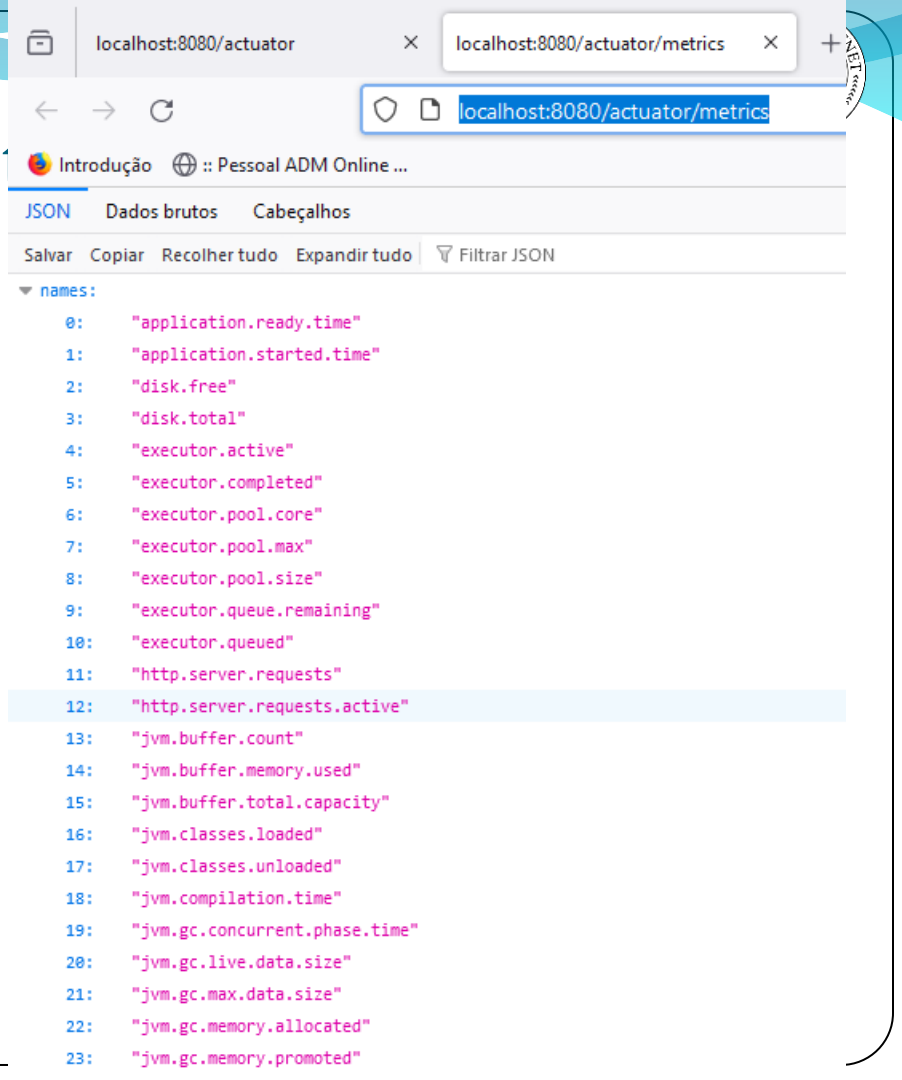
```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "caches-cache": {
      "href": "http://localhost:8080/actuator/caches/{cache}",
      "templated": true
    },
    "caches": {
      "href": "http://localhost:8080/actuator/caches",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8080/actuator/health/{*path}",
      "templated": true
    },
    "info": {
      "href": "http://localhost:8080/actuator/info",
      "templated": false
    },
    "conditions": {
      "href": "http://localhost:8080/actuator/conditions",
      "templated": false
    },
    "configprops": {
      "href": "http://localhost:8080/actuator/configprops",
      "templated": false
    },
    "configprops-prefix": {}
  }
}
```

SpringBoot Actuator

- Vamos ver as métricas:
- <http://localhost:8080/actuator/metrics>

SpringBoot

<http://localhost:8080/actuator/metrics/disk.total>



The screenshot shows a web browser window with two tabs: 'localhost:8080/actuator' and 'localhost:8080/actuator/metrics'. The address bar shows the URL 'localhost:8080/actuator/metrics'. The page content displays a list of metrics under the heading 'names:'. The metrics are listed as follows:

- 0: "application.ready.time"
- 1: "application.started.time"
- 2: "disk.free"
- 3: "disk.total"
- 4: "executor.active"
- 5: "executor.completed"
- 6: "executor.pool.core"
- 7: "executor.pool.max"
- 8: "executor.pool.size"
- 9: "executor.queue.remaining"
- 10: "executor.queued"
- 11: "http.server.requests"
- 12: "http.server.requests.active"
- 13: "jvm.buffer.count"
- 14: "jvm.buffer.memory.used"
- 15: "jvm.buffer.total.capacity"
- 16: "jvm.classes.loaded"
- 17: "jvm.classes.unloaded"
- 18: "jvm.compilation.time"
- 19: "jvm.gc.concurrent.phase.time"
- 20: "jvm.gc.live.data.size"
- 21: "jvm.gc.max.data.size"
- 22: "jvm.gc.memory.allocated"
- 23: "jvm.gc.memory.promoted"

SpringBoot Actuator

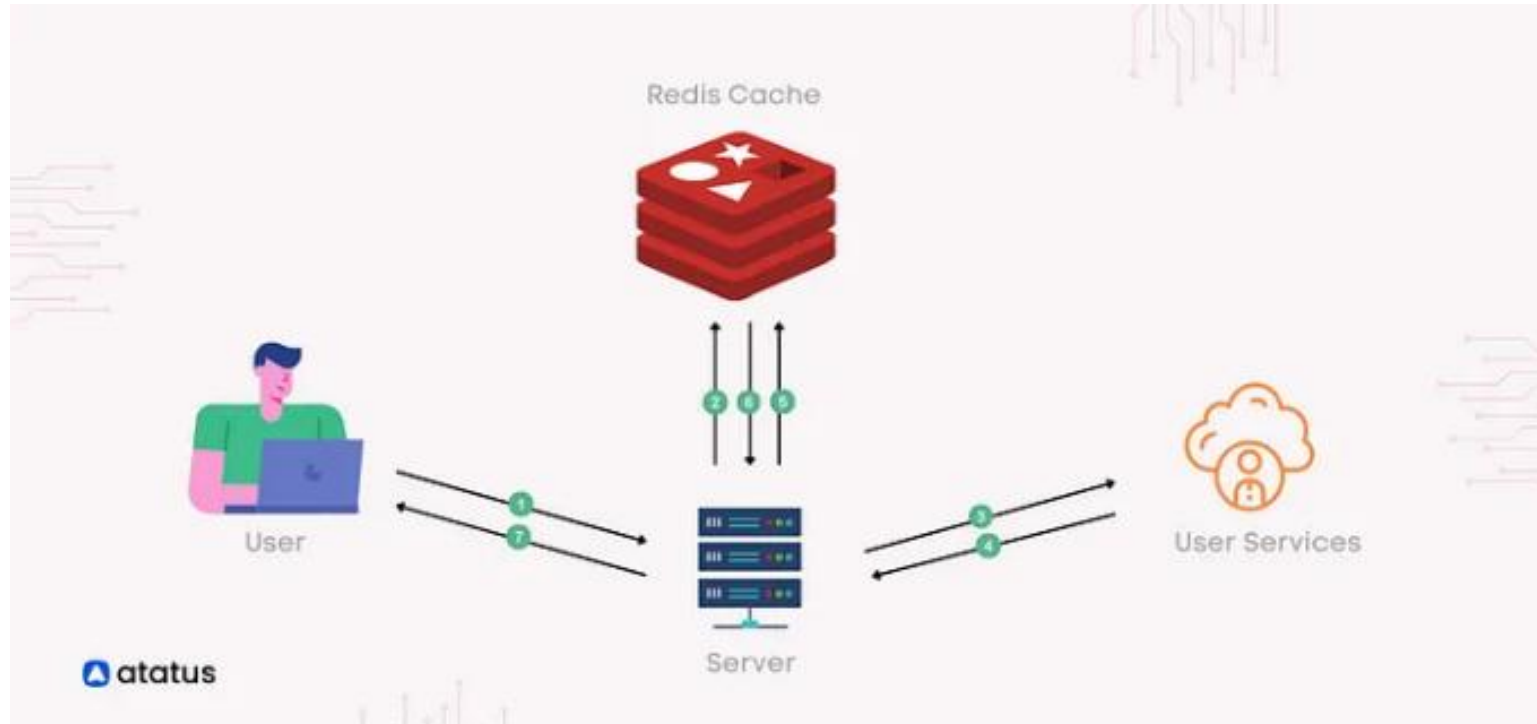
- Uma das vantagens do SpringBoot Actuator é permitir o monitoramento da aplicação para não permitir que a aplicação pare de funcionar.

Configurações de Serviços com Redis

Configurações de Serviços com Redis

- O Redis é um banco de dados normalmente usado como um armazenamento de dados na memória para compartilhar o estado entre instâncias de um serviço, armazenar em cache e intermediar mensagens entre serviços.
- Como todos os principais bancos de dados, o Redis faz mais, porém o foco desta aula é simplesmente usar o Redis para armazenar e recuperar da memória informações solicitadas.

Configurações de Serviços com Redis



Configurações de Serviços com Redis

- O Redis é um armazenamento de estrutura de dados de chave-valor de código aberto e na memória.
- O Redis oferece um conjunto de estruturas versáteis de dados na memória que permite a fácil criação de várias aplicações personalizadas.
- Os principais casos de uso do Redis incluem cache, gerenciamento de sessões, PUB/SUB e classificações.

Configurações de Serviços com Redis

- É o armazenamento de chave-valor mais conhecido atualmente.
- Ele tem a licença BSD, é escrito em código C otimizado e é compatível com várias linguagens de desenvolvimento.
- Redis é um acrônimo de REMote DIctionary Server (servidor de dicionário remoto).

Configurações de Serviços com Redis

- Por conta da sua velocidade e facilidade de uso, o Redis é uma escolha em alta demanda para aplicações web e móveis, como também de jogos, tecnologia de anúncios e IoT, que exigem o melhor desempenho do mercado.

Configurações de Serviços com Redis

- Ao utilizar o Redis, você reduz a carga no banco de dados principal, acelera o acesso a dados frequentemente acessados e melhora a escalabilidade horizontal de sua aplicação, resultando em tempos de resposta mais rápidos e uma experiência do usuário aprimorada.

Configurações de Serviços com Redis

- Desempenho muito rápido:
 - Todos os dados do Redis residem na memória principal do seu servidor, em contraste com a maioria dos sistemas de gerenciamento de banco de dados que armazenam dados em disco ou SSDs.
 - Ao eliminar a necessidade de acessar discos, bancos de dados na memória, como o Redis, evitam atrasos de tempo de busca e podem acessar dados com algoritmos mais simples que usam menos instruções de CPU.
 - Operações comuns exigem menos do que 10 milissegundos para serem executadas.

Configurações de Serviços com Redis

- Estruturas de dados na memória
 - O Redis permite que os usuários armazenem chaves que fazem o mapeamento para vários tipos de dados.
 - O tipo de dados fundamental é uma string, que pode ser em formato de dados de texto ou binários e ter no máximo 512 MB.
 - O Redis também é compatível com listas de strings na ordem em que foram adicionadas, conjuntos de strings não ordenadas, conjuntos classificados ordenados de acordo com uma pontuação, hashes que armazenam uma lista de campos e valores e HyperLogLogs para contar os itens exclusivos de um conjunto de dados.
 - Praticamente qualquer tipo de dados pode ser armazenado na memória usando o Redis.

Configurações de Serviços com Redis

- Versatilidade e facilidade de uso
 - O Redis é disponibilizado com várias ferramentas que tornam o desenvolvimento e as operações mais rápidas e fáceis, inclusive o PUB/SUB para publicar mensagens nos canais que são entregues para os assinantes.
 - O que é ótimo para sistemas de mensagens e chat, as chaves com TTL podem ter um tempo de vida útil determinado, após a qual elas se autoexcluem, o que ajuda a evitar sobrecarregar o banco de dados com itens desnecessários.

Configurações de Serviços com Redis

- Replicação e persistência
 - O Redis emprega uma arquitetura no estilo mestre/subordinado e é compatível com a replicação assíncrona em que os dados podem ser replicados para vários servidores subordinados.
 - Isso pode disponibilizar desempenho de leitura melhorado (à medida que as solicitações podem ser divididas entre os servidores) e recuperação quando o servidor primário passar por uma interrupção.
 - Para disponibilizar durabilidade, o Redis oferece compatibilidade com snapshots point-in-time (copiando o conjunto de dados do Redis no disco) e criando um Append Only File (AOF) para armazenar cada alteração de dados no disco conforme elas vão sendo gravadas.
 - Os dois métodos permitem a restauração rápida dos dados do Redis no caso de uma interrupção.

Configurações de Serviços com Redis

- Casos de uso do Redis:
 - **Armazenamento em cache:**
 - O Redis inserido na "frente" de outro banco de dados cria um cache na memória com excelente desempenho para diminuir a latência de acesso, aumentar o throughput e facilitar a descarga de um banco de dados relacional ou NoSQL.

Configurações de Serviços com Redis

- Casos de uso do Redis:
 - **Gerenciamento de sessões:**
 - O Redis é altamente indicado para tarefas de gerenciamento de sessões.
 - Basta usar o Redis como um armazenamento de chave-valor com o tempo de vida (TTL) correto nas chaves de sessão para gerenciar suas informações de sessão.
 - O gerenciamento de sessões é comumente exigido para aplicações on-line, como jogos, sites de comércio eletrônico e plataformas de mídia social.

Configurações de Serviços com Redis

- Casos de uso do Redis:
 - **Classificações em tempo real:**
 - Ao usar a estrutura de dados Sorted Set do Redis, os elementos são mantidos em uma lista, classificada de acordo com suas pontuações. Isso facilita a criação de classificações dinâmicas para mostrar quem está vencendo o jogo ou publicando as mensagens mais curtidas ou qualquer outra coisa que demonstre quem está na liderança.



Project

☐ Gradle - Groovy

☐ Gradle - Kotlin ☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 3.3.0 (SNAPSHOT)

☐ 3.3.0 (RC1)

☐ 3.2.6 (SNAPSHOT)

☒ 3.2.5

☐ 3.1.12 (SNAPSHOT)

☐ 3.1.11

Project Metadata

Group

Artifact

Name

Description

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Lombok DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

Spring Data Reactive Redis NOSQL

Access Redis key-value data stores in a reactive fashion with Spring Data Redis.

GENERATE CTRL + G

EXPLORE CTRL + SPACE

SHARE...